

Jenkins CI/CD — Terminology & Setup Notes

Personal reference guide, with navigation diagrams showing exactly where each setting lives

Contents

1. CI/CD basics
2. Item / Project Type (Freestyle vs Pipeline)
3. Source Code Management → Git
4. Build Setup (Freestyle) → Execute Shell
5. Build Triggers — Build Periodically & Poll SCM (Cron)
6. Webhook in GitHub
7. Plugins (e.g. Maven)
8. Post-Build Action → Static Analysis (PMD)
9. Pipeline & Declarative Pipeline Syntax

1. CI/CD

CI/CD = Continuous Integration / Continuous Delivery.

- **Continuous Integration** — developers frequently merge code into a shared repo; each merge is automatically built and tested.
- **Continuous Delivery** — the tested build is automatically prepared/released so it can be deployed at any time.

2. Item / Project Type

When creating a new job in Jenkins, you choose a type: **Freestyle** or **Pipeline**.

- **Freestyle** — configured entirely through the UI (point-and-click steps).
- **Pipeline** — configured as code (Groovy), usually stored as a `Jenkinsfile` alongside your source.

JENKINS DASHBOARD

Creating a Job: Item / Project Type

Dashboard → New Item → Enter name

Select type: Freestyle project / Pipeline → OK

Freestyle = simple point-and-click job. Pipeline = job defined as Groovy code (Jenkinsfile).

Where to find it: Dashboard → New Item → choose Freestyle project or Pipeline.

3. Source Code Management → Git

In a Freestyle job's configuration, the **Source Code Management** section is where you point Jenkins at your Git repository — the repo URL, credentials, and which branch to build.

JOB CONFIGURE

Source Code Management → Git

Source Code Mgmt

() None (•) Git

Repository URL

https://github.com/user/repo.git

Credentials

- none - / add

Branch Specifier

*/main

Job → Configure → Source Code Management → Git. Fill in the repository URL and branch specifier.

4. Build Setup (Freestyle) → Execute Shell

Under **Build Steps**, add a build step of type **Execute shell** and enter the shell commands to run during the build (e.g. `uptime` , `whoami`).

JOB CONFIGURE (FREESTYLE)

Build Steps → Add build step → Execute shell

Build Step

Execute shell

Command

uptime

Command

whoami

Job → Configure → Build Steps → Add build step → Execute shell.

5. Build Triggers — Build Periodically & Poll SCM

- **Build Periodically** — builds on a cron schedule regardless of whether the repo changed.
- **Poll SCM** — checks the repo on a cron schedule and only builds if there was a change.

Cron format: * * * * * → MINUTE HOUR DAY_OF_MONTH MONTH DAY_OF_WEEK

JOB CONFIGURE

Build Triggers

Trigger ☒ Build periodically

Schedule (cron)

Trigger ☒ Poll SCM

Schedule (cron)

Cron format: MINUTE HOUR DAY_OF_MONTH MONTH DAY_OF_WEEK. 'Build periodically' always builds on schedule; 'Poll SCM' only builds if the repo changed.

Job → Configure → Build Triggers. Tick "Build periodically" or "Poll SCM" and enter the cron schedule.

Jenkins also keeps a **Git Poll Log** per job, showing the history/result of each SCM poll — useful for confirming polling is actually running and whether it detected changes.

6. Webhook in GitHub

Instead of Jenkins polling the repo on a timer, GitHub can actively notify Jenkins the moment a commit is pushed. This needs configuration on **both** sides:

6a. On the Jenkins job

Enable `GitHub hook trigger for GITScm polling` under Build Triggers.

JOB CONFIGURE → BUILD TRIGGERS

GitHub hook trigger for GITScm polling

Trigger ☒ GitHub hook trigger for GITScm polling

Tick this box so Jenkins listens for a push notification from GitHub's webhook instead of only polling on a timer.

Job → Configure → Build Triggers → GitHub hook trigger for GITScm polling.

6b. On the GitHub repository

Add a webhook pointing at your Jenkins server so GitHub calls Jenkins on every push.

GITHUB REPOSITORY

Repo → Settings → Webhooks → Add webhook

Payload URL	http://<jenkins-ip>:8080/github-webhook/
Content type	application/json
Secret	(optional)
Which events?	Just the push event
Active	☒ ✓ enabled
Add webhook	

GitHub repo → Settings → Webhooks → Add webhook. Payload URL: `http://<jenkins-ip>:8080/github-webhook/` , Content type: `application/json` .

Result: any commit/change pushed to the GitHub repo fires the webhook, which notifies Jenkins to start the build immediately — no waiting for the next poll.

7. Plugins

A **Plugin** extends what Jenkins can do — e.g. building/running code with tools like **Maven**, integrating Git, or generating static-analysis reports.

7a. Installing a plugin

MANAGE JENKINS

Installing a Plugin

Manage Jenkins → Plugins → Available plugins → search e.g. "Warnings"

check box → Install

Plugins add extra build/analysis capability to Jenkins, e.g. Maven Integration, Git, Warnings Next Generation (for PMD/static-analysis reports).

Manage Jenkins → Plugins → Available plugins → search (e.g. "Warning") → check the box → Install.

7b. Configuring a tool (e.g. Maven)

Once the Maven Integration plugin is installed, register one or more Maven installations under Global Tool Configuration. You can register multiple versions here, each under its own name.

Maven Installations → Add Maven

Name

abcmaven

Install automatically

✓ enabled

Version

3.9.x (choose from dropdown)

You can add multiple Maven versions here, each with its own name, and pick the one you need per job.

Manage Jenkins → Tools → Maven Installations → Add Maven. Give it a name (e.g. "abcmaven") and pick a version.

7c. Using Maven in a job's build step

In the Freestyle job, add a build step that invokes the named Maven installation with a goal (e.g. `package`), and note the workspace path where its report output lands (e.g. `Codereview/target/pmd.xml`).

Build Steps → Invoke top-level Maven targets

Maven Version

abcmaven

Goals

package

Report path

workspace/Codereview/target/pmd.xml

Job → Configure → Build Steps → Invoke top-level Maven targets. Maven Version: your named installation (e.g. "abcmaven"); Goal: package.

8. Post-Build Action → Static Analysis (PMD)

After the build runs, a **Post-Build Action** can record and graph static-analysis results, e.g. PMD output, giving a graphical trend of code-quality issues over time.

Post-build Actions → Record compiler warnings and static analysis results

Add post-build action

Record compiler warnings and static analysis results

Tool

PMD

Report file pattern

**/pmd.xml

Produces a graphical trend/analysis of static-analysis (PMD) results on the job page.

Job → Configure → Post-build Actions → Record compiler warnings and static analysis results → Tool: PMD → Report file pattern: `**/pmd.xml`.

9. Pipeline

Pipeline = a set of tasks/stages executed one after another, defined as code (Groovy-based) rather than clicked together in the UI.

Declarative Pipeline fundamentals

Below is the sample declarative pipeline from the notes, showing `agent` , `tools` , `parameters` (string, text, boolean, choice, password) and `stages` (including a conditional `when` stage):

```
pipeline {
    agent any

    tools {
        //maven 'abcmaven'
    }
    parameters {
        string(name: 'tool', defaultValue: 'Mr Jenkins', description: 'ToolName')

        text(name: 'BIOGRAPHY', defaultValue: '', description: 'Enter some information about the person')

        booleanParam(name: 'TOGGLE', defaultValue: true, description: 'Toggle this value')

        choice(name: 'CHOICE', choices: ['One', 'Two', 'Three'], description: 'Pick something')

        password(name: 'PASSWORD', defaultValue: 'SECRET', description: 'Enter a password')
    }
    stages {
        stage('Build') {
            steps {
                //
            }
        }
        stage('Test') {
            steps {
                //
            }
        }
        stage('DEV') {
            when {
                expression { params.ENV == 'DEV' }
            }
            steps {
                //
            }
        }
        stage('Example') {
            steps {
                echo "Hello ${params.tool}"

                echo "Biography: ${params.BIOGRAPHY}"

                echo "Toggle: ${params.TOGGLE}"

                echo "Choice: ${params.CHOICE}"

                echo "Password: ${params.PASSWORD}"
            }
        }
    }
}
```

Quick read of the structure:

- `agent any` — run on any available Jenkins agent.
- `tools { }` — declare named tools (e.g. a Maven installation) to make available on PATH for this pipeline.
- `parameters { }` — build-time inputs a user fills in when triggering the job (string, multi-line text, boolean toggle, dropdown choice, masked password).
- `stages { }` — the ordered list of stages; each `stage` has `steps` that actually run, and can optionally have a `when { expression { ... } }` block to run conditionally.

Note: the screenshots in this guide are simplified illustrative diagrams (not vendor screenshots) built to show exactly which menu path and fields to use in the real Jenkins/GitHub UI, since the actual UI may vary slightly by version.